

FastAPI Backend Course Cheat Sheets and Exercises

Python OOP (Classes, Inheritance, Polymorphism)

- **Class Definition:** In Python, a *class* bundles data (attributes) and functions (methods) into a new object type. Creating `class MyClass:` defines a blueprint; instantiating it (`obj= MyClass()`) makes an object.
- **Attributes & Methods:** Class attributes store state; methods (defined in the class) manipulate that state. Example:

```
class Animal:
    species = "Unknown"      # class attribute
    def __init__(self, name):
        self.name = name     # instance attribute
    def speak(self):
        print(f"{self.name} makes a sound")
```

- **Inheritance:** Classes support inheritance. A derived class can extend or override its base classes' behavior. Python allows multiple inheritance. Example:

```
class Dog(Animal):           # inherits from Animal
    def speak(self):         # override method
        print(f"{self.name} says woof")
```

- **Polymorphism:** Different classes can implement the same method name with different behavior. For example, calling `speak()` on an `Animal` vs `Dog` object invokes different code (method overriding). This is polymorphism in action.
- **Constructor (`__init__`):** Initialization logic runs in `__init__`. Use `self` to refer to instance attributes.
- **Example:**

```
animal = Animal("Generic")
dog = Dog("Fido")

animal.speak()    # -> "Generic makes a sound"
dog.speak()       # -> "Fido says woof"
```

Reference: Python's official tutorial explains that classes bundle data and behavior and support inheritance with method overriding.

Exercise (OOP Review)

1. **Define Classes:** Create a class `Vehicle` with an attribute `wheels`. Inherit from it to make `Car` and `Bike`. Override a method `describe()` in each subclass to print the number of wheels.

Solution (Python):

```
class Vehicle:
    def __init__(self, wheels):
        self.wheels = wheels
    def describe(self):
        print(f"Vehicle with {self.wheels} wheels.")

class Car(Vehicle):
    def describe(self):
        print(f"Car: {self.wheels} wheels, suitable for road.")

class Bike(Vehicle):
    def describe(self):
        print(f"Bike: {self.wheels} wheels, human-powered.")

# Test
car = Car(4)
bike = Bike(2)
car.describe()    # Car: 4 wheels, suitable for road.
bike.describe()   # Bike: 2 wheels, human-powered.
```

REST API & FastAPI Basics

- **REST & HTTP Methods:** A REST API uses HTTP endpoints (paths) and methods. The main methods are GET (fetch data), POST (create), PUT/PATCH (update), DELETE (remove). Each endpoint URL is called a *path operation*.
- **JSON Responses:** FastAPI auto-serializes returned Python data (dicts, lists) to JSON. By default it uses `application/json` content type.
- **Creating a FastAPI App:** Begin by instantiating `FastAPI()` and defining routes with decorators :

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/")
async def read_root():
    return {"message": "Hello World"}
```

This creates a GET endpoint at `/` that returns JSON. As shown above, `app = FastAPI()` and `@app.get("/")` are the typical starting steps.

- **Running the Server:** Use Uvicorn (ASGI server). For example:

```
fastapi dev main.py
```

This starts the server on `http://127.0.0.1:8000`. The `--reload` flag auto-restarts on code changes.

- **Automatic Docs:** FastAPI generates interactive docs. Visit `/docs` for Swagger UI and `/redoc` for ReDoc interface. These show all routes and schemas automatically (no extra work needed).
- **Path vs Query Parameters:**
 - *Path parameters* are part of the URL. Example: `@app.get("/items/{item_id}")` will capture that part as an argument to the function.
 - *Query parameters* come after `?` in the URL and can be declared as function parameters with defaults.
- **Response Model & Status Codes:** You can specify response models (using Pydantic) and status codes. By default, FastAPI returns 200 OK for successful returns.

Reference: FastAPI's tutorial shows that the simplest app uses `FastAPI()` and `@app.get()` to create endpoints, and it auto-generates `/docs` (Swagger) for you.

Exercise (Basic FastAPI Routing)

1. **Hello Endpoint:** Create a FastAPI app with a GET endpoint `/hello` that returns `{"msg": "Hello, FastAPI!"}`.

Solution (Python):

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/hello")
async def say_hello():
    return {"msg": "Hello, FastAPI!"}
```

1. **Path Parameter:** Add an endpoint `GET /items/{item_id}` that returns `{"item_id": <item_id>}`.

Solution:

```
@app.get("/items/{item_id}")
async def read_item(item_id: int):
    return {"item_id": item_id}
```

Pydantic Models and Validation

- **BaseModel:** FastAPI uses Pydantic models to define the expected data schema. A model is a class inheriting from `BaseModel`. Fields are declared as annotated class attributes.
- **Validation:** When a request includes a JSON body matching a Pydantic model, FastAPI auto-validates and converts types. If a field is missing or the wrong type, FastAPI returns a 422 error automatically.
- **Example Model:**

```
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    price: float
    in_stock: bool = True
```

Here name and price are required; in_stock is optional with default True.

- **Usage in Endpoints:** Declare a model as a function parameter to a POST/PUT endpoint, and FastAPI will parse the request body into that model. For example:

```
@app.post("/items/")
async def create_item(item: Item):
    return {"item": item}
```

This returns the parsed item back as JSON.

- **Type Coercion:** Pydantic will try to coerce types. For example, if JSON sends "price": "9.99", Pydantic will convert it to a float 9.99.
- **Accessing Fields:** You can access model fields like attributes (`item.name`). Use `item.dict()` or `item.model_dump()` to get a dict for serialization.
- **Nested Models:** You can nest Pydantic models inside each other to represent complex data.
- **Validation Error:** If validation fails, FastAPI returns a 422 response with details of the error. For example, sending a string for `price` when it should be a number yields an error message in JSON.

Reference: Pydantic docs explain that models are classes inheriting from `BaseModel` and defining typed fields. After creation, Pydantic “guarantees that the fields of the resultant model instance will conform to the field types”. The example also shows type coercion (`user.id` becomes `int`).

Exercise (Pydantic Validation)

1. **User Model:** Define a Pydantic model `User` with fields `username: str` and `age: int`. Create a POST endpoint `/users/` that accepts a `User` and returns it.

Solution:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
class User(BaseModel):
    username: str
    age: int

@app.post("/users/")
async def create_user(user: User):
    return {"user": user}
```

1. **Test Validation:** What happens if you call `/users/` with body

```
{"username": "alice", "age": "not a number"}
```

Solution: FastAPI returns HTTP 422 with detail saying `value is not a valid integer`. The request is rejected by validation.

2. **Default Value:** Modify `User` so that `email: str = "unknown@example.com"` (default). Verify that omitting `email` uses the default.

Authentication & JWT

- **Security Overview:** FastAPI supports OAuth2 and JWT for securing endpoints. A common pattern is OAuth2 password flow with a JWT bearer token.
- **JWT Explained:** JSON Web Tokens (JWT) are compact URL-safe JSON tokens. They contain claims (like user ID) and are signed to prevent tampering. FastAPI's docs note JWT is a standard to encode JSON into a string.
- **Creating Tokens:** Typically use `PyJWT` or FastAPI's utilities:

```
from datetime import datetime, timedelta
import jwt # PyJWT

SECRET = "your-secret-key"
data = {"sub": "user1"} # subject (e.g. username)
to_encode = data.copy()
expire = datetime.utcnow() + timedelta(minutes=30)
to_encode.update({"exp": expire})
token = jwt.encode(to_encode, SECRET,
    algorithm="HS256")
```

- **Using in FastAPI:** You might create a login endpoint that checks credentials, then returns `{"access_token": token, "token_type": "bearer"}`. Other endpoints use `Depends(oauth2_scheme)` to enforce token auth (from `fastapi.security`).
- **Token Verification:** In protected endpoints, decode the JWT (with the same secret) to verify validity. PyJWT's `jwt.decode(token, SECRET, algorithms=["HS256"])` will raise an error if invalid or expired.
- **Detail vs Flask:** FastAPI's `HTTPException` (for auth errors) can include JSON in the detail field, unlike Starlette's (only strings).
- **Example:** FastAPI docs show `raising HTTPException(status_code=401, detail="Nope!")` for unauthorized attempts.
- **Summary:** In short, use OAuth2 password flow, generate a signed JWT on login, send it in `Authorization: Bearer <token>`, and protect other routes by verifying the token.

Reference: The FastAPI docs describe OAuth2 with JWT tokens (standard security approach). They note "JWT means 'JSON Web Tokens'... it's a standard to codify a JSON object in a long string". FastAPI's `HTTPException` is used to signal auth errors and can accept JSON data in its detail field.

Exercise (JWT Auth)

1. **Generate JWT:** Write Python code to create a JWT token with claim `{"sub": "testuser"}` that expires in 1 hour, using secret `"abc123"`.

Solution:

```
from datetime import datetime, timedelta
import jwt

SECRET = "abc123"
payload = {"sub": "testuser"}
payload.update({"exp": datetime.utcnow() + timedelta(hours=1)})
token = jwt.encode(payload, SECRET, algorithm="HS256")
print(token)  # This is the JWT
```

1. **Decode JWT:** Using PyJWT, decode the token back to verify it.

Solution:

```
decoded = jwt.decode(token, SECRET, algorithms=["HS256"])
print(decoded)  # {'sub': 'testuser', 'exp': ...}
```

1. **FastAPI Protected Route (outline):** Sketch a FastAPI endpoint `/secret` that requires a JWT and raises 401 if invalid. (You don't need to write full security code, just show raising `HTTPException` on failure.)

Solution Outline:

```
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

@app.get("/secret")
async def read_secret(token: str = Depends(oauth2_scheme)):
    try:
        jwt.decode(token, SECRET, algorithms=["HS256"])
    except jwt.PyJWTError:
        raise HTTPException(status_code=401, detail="Invalid auth")
    return {"secret_data": "42"}
```

CORS & Security Settings

- **CORS(Cross-Origin):** Browser restricts AJAX calls to your API's origin. Use CORS middleware to allow specific origins. FastAPI (via Starlette) provides `CORSMiddleware`.
- **Enabling CORS:**

```
from fastapi.middleware.cors import CORSMiddleware
app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://example.com", "http://localhost:3000"],      #
    origins_allowed
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

This configures which origins can call your API. Without it, cross-origin requests from browsers will be blocked.

- **FastAPI Docs:** The docs show using `CORSMiddleware` for CORS. You must specify `allow_origins`. Setting `["*"]` allows any origin (useful for development).
- **Other Security:** FastAPI also supports OAuth2 scopes, basic auth, etc. You can also configure JWT in docs, and use `Depends` to enforce OAuth2 flow on routes.

Reference: FastAPI's security tutorials and docs explain CORS usage. The docs example shows configuring `CORSMiddleware` to allow cross-origin requests.

Exercise (CORS)

1. **Enable CORS:** Modify a FastAPI app to allow all origins and all methods.

Solution:

```
from fastapi.middleware.cors import CORSMiddleware
app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],          # allow all
    allow_methods=["*"],         # GET, POST, etc.
    allow_headers=["*"],
    allow_credentials=True
)
```

1. **Test CORS:** Explain what would happen if a web page at `http://evil.com` tries to fetch from your API without CORS enabled.

Solution: The browser would block the request due to CORS policy, and the JavaScript would receive a CORS error. Enabling CORS for that origin or all origins is needed to allow it.

Async Concurrency (async/await)

- **Async Path Functions:** FastAPI supports asynchronous endpoint functions. Use `async def` for your path operation functions when doing I/O-bound work.
- **Await Inside:** You can only use `await` inside an `async def` function. If you have async libraries (e.g. database drivers, HTTP clients), you should await them.
- **Mixing async and sync:** FastAPI allows mixing normal `def` and `async def`. If you use `def` (sync), FastAPI will run that function in a threadpool so it doesn't block the event loop. However, best practice for I/O-heavy endpoints is `async def`.
- **Concurrency Model:** Python's `asyncio` event loop runs `async` functions concurrently (non-blocking). For example, two `async def` endpoints can await on I/O (like `await asyncio.sleep(1)`) concurrently without blocking each other.
- **Parallelism vs Concurrency:** For CPU-bound tasks, `asyncio` is not helpful; use normal functions or offload to threads/processes.
- **Code Example:**

```
import asyncio

@app.get("/wait")
async def wait_seconds(sec: int):
    await asyncio.sleep(sec)
    return {"slept": sec}
```

This endpoint asynchronously sleeps without blocking other requests.

- **Concurrency Note:** The FastAPI docs emphasize: declare your functions as `async def` and use `await`. If you don't need `await`, you can still use `async def`.

Reference: FastAPI's "Concurrency and async/await" section shows how to declare endpoints with `async def` and the need to use `await` within them. It also notes that FastAPI handles sync functions in a threadpool and recommends `async def` for non-blocking I/O.

Exercise (Async Endpoint)

1. **Async Sleep:** Write an endpoint `GET /delay/{seconds}` that waits the given number of seconds asynchronously and returns a message.

Solution:

```
import asyncio

@app.get("/delay/{seconds}")
async def delay(seconds: int):
    await asyncio.sleep(seconds)
    return {"delayed": seconds}
```

1. **Concurrent Requests:** If two clients call `/delay/2` at the same time, explain how FastAPI handles this.

Solution: Both requests will `await asyncio.sleep(2)` concurrently. Since sleep is non-blocking, the server can handle both and respond after ~2 seconds total. They run concurrently (not sequentially).

HTTP Error Handling

- **Raising Errors:** Use `fastapi.HTTPException` to return error responses from your code. For example:

```
from fastapi import HTTPException

@app.get("/items/{id}")
async def read_item(id: int):
    if id not in db:
        raise HTTPException(status_code=404, detail="Item not found")
    return {"id": id, "value": db[id]}
```

This sends a 404 Not Found with a JSON body `{"detail": "Item not found"}`.

- **Built-in Exceptions:** FastAPI's `HTTPException` inherits Starlette's and allows any JSON-serializable detail. The docs note the only difference is FastAPI's can include JSON in detail (Starlette only allowed strings).
- **Validation Errors:** If request validation fails (e.g. wrong type), FastAPI automatically returns a 422 with details about which field was invalid. You can also catch these with custom handlers if needed.
- **Custom Handlers:** You can override exception handlers. For example, to return plain text on 404 instead of JSON, you can register an exception handler for `StarletteHTTPException`. (Advanced topic.)
- **Status Codes:** Besides raising, you can return custom status with `return JSONResponse(status_code=..., content=...)` or with `@app.post(..., status_code=201)`.
- **Example:** The docs example raises `HTTPException(status_code=418, detail="Nope!")` if `item_id == 3`.

Reference: FastAPI's docs show using `raise HTTPException` to handle error cases and explain FastAPI's `HTTPException` class. They demonstrate raising a 418 status code with a detail message.

Exercise (Error Handling)

1. **404 for Missing:** Modify a GET `/users/{user_id}` endpoint so it returns 404 if `user_id` is not in your fake database.

Solution:

```
@app.get("/users/{user_id}")
async def get_user(user_id: int):
    if user_id not in users_db:
        raise HTTPException(status_code=404, detail="User not found")
    return users_db[user_id]
```

1. **Raise 400:** Write an endpoint that raises a 400 error if a query param `q` is missing or empty.

Solution:

```
from fastapi import Query

@app.get("/search")
async def search(q: str = Query(..., min_length=1)):
    if not q:
        raise HTTPException(status_code=400, detail="Query cannot be empty")
    return {"results": []}
```

Database (PostgreSQL via ORM)

- **ORM Models:** Use an ORM (SQLAlchemy or FastAPI's SQLAlchemy) to define database tables as Python classes. For example, with SQLAlchemy (built on SQLAlchemy and Pydantic):

```
from sqlalchemy import SQLAlchemy, Field

class Hero(SQLAlchemy, table=True):
    id: int = Field(default=None, primary_key=True)
    name: str
    secret_name: str
```

This defines a table with columns `id`, `name`, `secret_name`.

- **Engine and Session:** Create a database engine connecting to PostgreSQL, e.g. `create_engine("postgresql://user:pass@localhost/db")`. Then create a session for DB operations. A common pattern is a session dependency in FastAPI:

```
from sqlalchemy import Session, create_engine

engine = create_engine("postgresql://user:pass@localhost/db")
SQLAlchemy.metadata.create_all(engine) # create tables

def get_session():
    with Session(engine) as session:
        yield session
```

- **CRUD Endpoints:** In endpoints, get the session via `Depends(get_session)`, then use SQLAlchemy to query. Example:

```
@app.post("/heroes/")
async def create_hero(hero: Hero, session: Session = Depends(get_session)):
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero
```

- **Transactions:** Remember to `commit()` changes (POST/PUT/DELETE). Queries (SELECT) work on the session as is.
- **Other Tools:** You can also use raw SQL (e.g. `psycopg2`) or higher-level libraries (Databases, Tortoise) but SQLAlchemy/SQLAlchemy is most common.
- **Note on SQLAlchemy:** The FastAPI tutorial uses SQLAlchemy (by the same author) which tightly integrates Pydantic and SQLAlchemy. You can use either; the key is defining models and using sessions.
- **Summary:** The ORM maps classes to tables. Define models (with fields) and create an engine. Use sessions to add/query/delete in your route functions.

Reference: FastAPI's tutorial on SQL databases uses SQLAlchemy (built on SQLAlchemy and Pydantic) and explains it's made by FastAPI's author. SQLAlchemy supports any SQLAlchemy database (like PostgreSQL). The docs show example code creating models, engine, and tables.

Exercise (Database Integration)

1. **Define Model:** Define a SQLAlchemy model `Product` with fields `id: int (PK)`, `name: str`, `price: float`.

Solution:

```
from sqlalchemy import SQLAlchemy, Field

class Product(SQLAlchemy, table=True):
    id: int = Field(default=None, primary_key=True)
    name: str
    price: float
```

1. **Create Engine:** Show code to create a PostgreSQL engine and tables.

Solution:

```
from sqlalchemy import create_engine

engine = create_engine("postgresql://user:pass@localhost/mydb")
SQLAlchemy.metadata.create_all(engine)
```

1. **CRUD Endpoint:** Sketch a POST endpoint `/products/` that inserts a new product into the DB and returns it. Assume a `get_session()` dependency as above.

Solution:

```
from fastapi import Depends
from sqlalchemy import Session

@app.post("/products/")
async def create_product(product: Product, session: Session =
    Depends(get_session)):
    session.add(product)
    session.commit()
    session.refresh(product)
    return product
```

3rd-Party APIs with requests

- **Requests Library:** Use Python's `requests` to call external HTTP APIs. It's simple: `requests.get(url)`, `requests.post(url,json=data)`, etc.
- **Example:**

```
import requests
response = requests.get("https://api.example.com/data")
data = response.json()
```

This sends a GET and parses the JSON response.

- **Parameters & Auth:** Add query params via `params={'q': 'search'}` or headers via `headers={'Authorization': 'Bearer ...'}` in the `requests.get/post` call.
- **Error Handling:** Check `response.status_code` for success (200-range) or errors. Use `response.raise_for_status()` to throw exceptions on HTTP errors.
- **In FastAPI Scripts:** You might use `requests` in a utility function or script. Within a FastAPI endpoint, prefer asynchronous HTTP clients (like `httpx`) to avoid blocking, but in simple scripts `requests` is fine.
- **Readability:** As docs state, "Requests allows you to send HTTP/1.1 requests extremely easily" without manual encoding.

Reference: The Requests documentation describes it as "an elegant and simple HTTP library for Python" that eliminates manual URL/query building.

Exercise (External API Call)

1. **Fetch Data:** Using `requests`, fetch JSON from `https://jsonplaceholder.typicode.com/todos/1` and print the `title`.

Solution:

```
import requests
res = requests.get("https://jsonplaceholder.typicode.com/todos/1")
res.raise_for_status()
todo = res.json()
print(todo["title"])    # prints the title of the todo
```

1. **FastAPI Proxy Endpoint:** Create a FastAPI endpoint `/external` that fetches the above JSON and returns it directly.

Solution:

```
@app.get("/external")
async def get_external():
    res = requests.get("https://jsonplaceholder.typicode.com/todos/1")
    res.raise_for_status()
    return res.json()
```

(Note: This is synchronous; in production prefer async HTTP client.)

CLI with argparse

- **Argparse Overview:** The `argparse` module in Python's standard library simplifies writing command-line interfaces. You create a parser, define expected arguments, and call `parse_args()`.

- **Basic Usage:**

```
import argparse

parser = argparse.ArgumentParser(description="Process some numbers")
parser.add_argument("numbers", nargs="+", type=int, help="list of numbers")
args = parser.parse_args()
print(sum(args.numbers))
```

Running this script with arguments (e.g. `python script.py 1 2 3`) will parse them into `args.numbers`.

- **Options vs Positional:** Positional args have no prefix. Options use `--name` or `-n`:

```
parser.add_argument("-v", "--verbose", action="store_true", help="verbose output")
```

- **Mutually Exclusive Groups and Subparsers:** Argparse allows grouping options and creating subcommands, but basic usage covers most needs.
- **Help Message:** Argparse auto-generates `-h/--help` usage info.
- **Integration:** Use `argparse` in standalone scripts (outside FastAPI) to automate tasks or run CLI tools.

Reference: As Real Python notes, Python's `argparse` "offers a comprehensive solution" for CLIs. By the end of their tutorial, you can parse arguments, define options, and generate help messages using `argparse`.

Exercise (Argparse CLI)

1. **Sum CLI:** Write a Python CLI that accepts any number of integers (positional) and prints their sum.
Solution (Python):

```
import argparse

parser = argparse.ArgumentParser(description="Sum integers")
parser.add_argument("ints", nargs="+", type=int, help="integers to sum")
args = parser.parse_args()
print(sum(args.ints))
```

Run as `python cli.py 3 5 2` to print 10.

1. **Options:** Extend the CLI to accept a `--verbose` flag that, if set, prints each number before summing.
Solution:

```
parser.add_argument("-v", "--verbose", action="store_true", help="print inputs")
args = parser.parse_args()
if args.verbose:
    print("Numbers:", args.ints)
print("Sum =", sum(args.ints))
```

Project Structure & Debugging

- **Modularity:** Organize code with separation of concerns:
- **main.py** (or `app/main.py`): creates FastAPI instance and includes routers.
- **Routers/Endpoints:** Use `APIRouter` to split routes into modules (e.g. `user_router`, `item_router`).
- **Models:** Put Pydantic and ORM models in a `models.py` or `schemas.py`.
- **Services/CRUD:** Business logic can be in `services` or `crud.py` modules.
- **Dependencies:** Functions (e.g. `get_session`) in `dependencies.py`.
- **Logging:** Use Python's `logging` module to log info/warnings. Example:

```
import logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
logger.info("Starting app")
```

- **Debugging:** You can use breakpoints (`import pdb; pdb.set_trace()`) or editor debugging tools. FastAPI's log output is helpful, and you can enable verbose logs in Uvicorn (`--log-level debug`).
- **Environment & Config:** Store secrets (DB URL, JWT secret) in environment variables or config files, not in code.
- **Comments and Doc:** Document your code. FastAPI also allows adding docstrings and descriptions in path operation decorators (which appear in the auto docs).

Testing FastAPI

- **TestClient:** FastAPI provides a `TestClient` (based on Starlette's) for testing. Use it with `pytest`.
- **Basic Test:**

```
from fastapi.testclient import TestClient
client = TestClient(app)

def test_read_root():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Hello World"}
```

In tests, you don't `await` the calls; just use normal calls on the client.

- **Structure:** Place tests in `tests/` directory or alongside modules (like `test_main.py`) and run with `pytest`.
- **Dependency Overrides:** For unit tests, you can override dependencies (e.g., database) using `app.dependency_overrides`.
- **Coverage:** Write tests for each endpoint (success and failure cases).

Reference: The FastAPI testing guide shows creating a `TestClient(app)` and writing normal `def test_*` functions with asserts. Their example checks status codes and JSON output.

Exercise (Testing)

1. **Hello Test:** Given the `/hello` endpoint above, write a pytest function to test it.

Solution:

```
from fastapi.testclient import TestClient
def test_say_hello():
    client = TestClient(app)
    response = client.get("/hello")
    assert response.status_code == 200
    assert response.json() == {"msg": "Hello, FastAPI!"}
```

1. **Error Test:** For the `/items/{item_id}` that returns 404 for missing items, write a test that confirms requesting a non-existent item yields 404.

Solution:

```
def test_item_not_found():
    client = TestClient(app)
    response = client.get("/items/999")
    assert response.status_code == 404
    assert response.json()["detail"] == "Item not found"
```

Summary of Teaching Plan

The above cheat sheets cover all topics listed in the syllabus. In class, you can **project or share the cheat sheets** with students and discuss the bullet points, then have students work on the exercises. Students should use the cheat sheets as reference while solving the exercises, promoting autonomy.

